
EVALUATING MODEL GENERALIZATION FOR COW DETECTION IN FREE-STALL BARN SETTINGS: INSIGHTS FROM THE COW LOCALIZATION (COLO) DATASET

A PREPRINT

 **Mautushi Das**

School of Animal Sciences

Virginia Tech

Blacksburg, VA 24061

mautushid@vt.edu

 **Gonzalo Ferreira**

School of Animal Sciences

Virginia Tech

Blacksburg, VA 24061

gonf@vt.edu

 **C. P. James Chen ***

School of Animal Sciences

Virginia Tech

Blacksburg, VA 24061

niche@vt.edu

May 24, 2025

ABSTRACT

1 Precision livestock farming (PLF) increasingly relies on advanced object localization techniques to
2 monitor livestock health and optimize resource management. This study investigates the general-
3 ization capabilities of object detection models for cow detection in indoor free-stall barn settings,
4 focusing on varying training data characteristics such as view angles and lighting, and model com-
5 plexities. Leveraging the newly released public dataset, COWs LOcalization (COLO) dataset, we
6 explore three key hypotheses: (1) Model generalization is equally influenced by changes in light-
7 ing conditions and camera angles; (2) Higher model complexity guarantees better generalization
8 performance; (3) Fine-tuning with custom initial weights trained on relevant tasks always brings
9 advantages to detection tasks. Our findings reveal considerable challenges in detecting cows in
10 images taken from side views and underscore the importance of including diverse camera angles in
11 building a detection model. Furthermore, our results emphasize that higher model complexity does
12 not necessarily lead to better performance. The optimal model configuration heavily depends on the
13 specific task and dataset, highlighting the need for careful model selection tailored to the particular
14 application. Lastly, while fine-tuning with transferred weights from related tasks can significantly

*Corresponding author: James Chen <niche@vt.edu>

benefit detection performance, especially when the source and target domains are closely aligned and the available labeled data is limited, this advantage diminishes as domain divergence increases or as more labeled data becomes available. In such cases, initializing with general pre-trained weights is often sufficient and more efficient, eliminating the need for labor-intensive task-specific weight initialization. Future work should focus on adaptive methods and advanced data augmentation to improve generalization and robustness. This study provides practical guidelines for PLF researchers on deploying computer vision models from existing studies, highlights generalization issues, and contributes the COLO dataset containing 1,254 images and 11,818 cow instances for further research.

Keywords Object detection · Cows · Model generalization · Model selection

1 Introduction

Precision Livestock Farming (PLF) increasingly relies on computer vision (CV) systems to monitor and manage livestock. Localizing individual animals from images and videos helps farm operators optimize health, behavior monitoring, and resource management. However, despite progress in applying CV to livestock settings, a critical challenge persists in deploying models across diverse environments. Inconsistent lighting, varied camera angles, and differences in farm layouts can significantly affect model generalization.

This study focuses on the generalization capabilities of YOLO-family and Real-Time Detection Transformer (RT-DETR) object detection models, specifically YOLOv11[1, 2], YOLOv12[3, 4], and RT-DETR[5, 6], in the task of localizing cows in free-stall barn settings. Specifically, we investigate how environmental factors and model complexities impact performance and explore whether fine-tuning with domain-specific data improves detection outcomes. We define three central hypotheses to guide this investigation:

- **Model generalization is equally influenced by changes in lighting conditions and camera angles.** Should camera angles prove more impactful than lighting conditions, it would be advisable to prioritize camera placement when deploying CV models in new environments.
- **Higher model complexity guarantees better generalization performance.** If a highly complex model does not ensure superior performance, future studies might consider adopting less computationally demanding models that still enhance performance.
- **The advantages of using fine-tuned models as initial training weights are persistent over pre-trained models.** If the advantages diminish as the training sample size increases in a similar cow localization task but different environments, the fine-tuning efforts may be deemed unnecessary when the deployment environment varies over multiple locations on a farm.

To facilitate these investigations, a public dataset named **COWs Localization (COLO)** will be developed and made available to the community. The findings of this study are expected to provide practical guidelines for PLF researchers on deploying CV models, considering available resources and anticipated performance.

2 Background Study

Object localization and its applications

Localizing livestock individuals from images or videos has become an essential task in PLF [7]. Such techniques allow farm operators to manage animal well-being and health in real-time, optimizing their resource management and improving sustainability [8, 9]. Technically speaking, in the field of CV, which is a subfield of artificial intelligence (AI) that focuses on translating visual information into actionable insights, localization tasks can be further categorized into object detection, object segmentation, and pose estimation. Object detection is the simplest form among these tasks,

localizing objects of interest by enclosing them within a rectangular bounding box defined by x and y coordinates, pixel width, and pixel height [10]. Successful instances in this category include YOLO (You Only Look Once) [11], Faster R-CNN (Region Convolutional Neural Networks) [12], and SSD (Single Shot MultiBox Detector) [13]. These models have been adopted and applied by animal scientists for detection in PLF. For example, a study [14] leveraged the DRN-YOLO model [15] to predict the eating behavior of dairy cows. This approach automates the assessment of feeding behavior, a critical indicator of cow health and productivity, and has saved labor efforts in complex farm settings. Another notable work is presented in [16], where the authors developed a posture detection system for pigs using deep learning models such as Faster R-CNN, SSD, and R-FCN, coupled with 2D imaging. This system accurately identifies standing and lying postures of pigs under commercial farm settings.

To achieve finer localization, object segmentation is employed to outline object contours pixel-wise, while pose estimation is performed by orienting and marking the key points of the object [17]. Some popular object segmentation models include Mask R-CNN [18], MS R-CNN [19], and U-Net [20]. This method of segmentation has also been applied in the field of PLF. In the study [21], the authors developed a method using Mask R-CNN [18] to segment and outline cattle in feedlots. Their technique enhances images and extracts key frames to accurately detect cattle, achieving superior precision with a mean pixel accuracy of 0.92. This supports advanced, real-time monitoring of cattle in PLF. Another study group [22] developed the PigMS R-CNN framework [19] to enhance the monitoring of group-housed pigs. This framework employs a 101-layer residual network along with a feature pyramid network and soft non-maximum suppression to effectively detect and segment pigs, thereby improving the accuracy of identifying and locating individual pigs in complex environments.

Model generalization, pre-training, and fine-tuning

Although implementing image-based systems in livestock production has become more common, current studies primarily focus on accuracy in homogenous environments and rarely address the challenges of model generalization. How a model can generalize to new environments is critical when farm operators deploy existing CV models in their own settings. Good generalization performance ensures that the model can reproduce similar results as reported in the original study, even in new environments with different conditions. Factors such as camera angles and the presence of occlusions can impact generalization in the deployment environment. Deploying the same model in a new environment does not necessarily guarantee the same performance as reported in the original study. Li et al. [23] also pointed out that the lighting conditions on farms in real applications can be highly variable, leading to poor generalization performance.

One explanation for poor generalization is the discrepancy between the pre-training process and the specific use case. Most CV models are released with pre-trained weights, obtained from training on a large-scale dataset. For example, the COCO dataset [24] is a general-purpose dataset containing over 200,000 images and a wide range of object categories, such as vehicles and household items. Directly deploying a model pre-trained on the COCO dataset to detect cows in a farm setting may not ensure satisfactory performance, as the dataset does not contain enough cow instances in different view angles or occlusions. To alleviate this discrepancy, fine-tuning is a common practice that modifies the prediction

head of the pre-trained model and updates the weights on a new dataset more relevant to the specific use case. Most application studies have adopted this approach to improve model generalization on their specific tasks [25, 26, 27].

Nevertheless, fine-tuning is not guaranteed to be successful, as the outcome depends on both the quantity and quality of the annotated dataset. For example, Zin et al. [28] deployed an object detection model to recognize cow ear tags in a dairy farm. Although the model achieved a high accuracy of 92.5% in recognizing the digits on the ear tags, more than 10,000 images were required for fine-tuning. Assembling such a large dataset is labor-intensive and requires specific training in annotating the images. The annotated dataset is rigorously organized in a specific format, such as COCO or YOLO, each with its own structure and coordinate systems, requiring professional annotation tools like Labelme [29], CVAT [30], or Roboflow [31].

Model characteristics

Prior to the introduction of YOLO, object detection methods generally relied on either the “sliding window with classifier” approach or the “region proposal with classifier” paradigm. The sliding window method involved applying a classifier across numerous locations and scales within an image, often requiring hundreds or even thousands of evaluations per image, which was computationally intensive and inefficient. On the other hand, advanced region proposal-based approaches divided the task into two steps: first, identifying potential object regions (i.e., region proposals) and then applying a classifier to these regions. In contrast, YOLO models are capable of performing object detection in a single pass through the network, which is why the acronym YOLO stands for “You Only Look Once.”

YOLOv11 introduces several architectural improvements to enhance both speed and accuracy. Key innovations include the C3K2 block (Cross Stage Partial with kernel size 2) for better feature extraction, Spatial Pyramid Pooling - Fast (SPPF) to improve multi-scale detection, and the Convolutional block with Parallel Spatial Attention (C2PSA) to help the model focus on important image regions. These enhancements make YOLOv11 more efficient and effective for tasks such as object detection, instance segmentation, pose estimation, and oriented object detection. Additionally, YOLOv11 is available in multiple sizes, ranging from lightweight models for edge devices to larger versions for high-performance applications [1, 2].

YOLOv12 takes a different approach by incorporating attention mechanisms to improve object detection. Instead of relying only on traditional Convolutional Neural Networks (CNNs), it leverages an optimized backbone (R-ELAN), 7×7 separable convolutions, and FlashAttention-driven area-based attention mechanisms [32, 33]. These features allow YOLOv12 to process images more efficiently, capture important details better, and perform well even in complex scenes. As a result, YOLOv12 achieves higher accuracy and faster inference speeds than its predecessors, making it ideal for real-time applications [3, 4].

RT-DETR (Real-Time Detection Transformer) is a transformer-based object detection model designed for speed and accuracy. Unlike traditional YOLO models, it does not require predefined anchor boxes or post-processing techniques like non-maximum suppression. Instead, RT-DETR directly predicts object locations and classes using a transformer

architecture. The attention-based intra-scale feature interaction (AIFI) module in RT-DETR enhances the model’s ability to understand the contextual relationships between objects and background elements in an image. Such characteristics make RT-DETR particularly effective for detecting small objects and handling occlusions, which are common challenges in real-world scenarios [5, 6].

This study included both CNN-based models (YOLOv11) and transformer-based architectures (YOLOv12 and RT-DETR) to balance speed, accuracy, and efficiency for a comprehensive evaluation of model generalization in object detection tasks.

Public datasets

A public dataset helps the community to develop methodology based on the same baseline. One famous example in CV is the ImageNet dataset [34], which serves as a benchmark for image classification. AlexNet [35], the winner of the ImageNet Large Scale Visual Recognition Challenge in 2012, demonstrated its outstanding capability to classify images in the ImageNet dataset using Rectified Linear Units (ReLU) as the activation function, rather than the traditional sigmoid function. The success of AlexNet accelerated the development of CV models in the following years, such as VGG [36], GoogLeNet [37], ResNet [38], and DenseNet [39]. However, similar to the challenges that pre-trained models face in specific use cases, a generic public dataset, such as ImageNet [34] and COCO [24], may not be sufficient for PLF applications.

There have been efforts to create public datasets for livestock scenarios. For example, the CattleEyeView dataset was collected to support applications like cattle pose estimation and behavior analysis, providing extensive annotations across 30,703 frames from top-down video sequences of cows [40]. Another study [41] leverages a public dataset for pigs comprising 3,600 images from 12 videos of group-housed pigs. The dataset is particularly designed for applications such as pig tracking. Additionally, the “OpenCows2020” dataset, developed by researchers from the University of Bristol, is a public dataset specifically designed for advancing non-intrusive monitoring of cattle. It supports precision farming applications such as automated productivity assessment, health and welfare monitoring, and veterinary research, including behavioral analysis and disease outbreak tracing. The dataset consists of 11,779 images with 13,026 labeled objects, mainly focusing on cattle [42]. While these publicly available datasets provide a solid foundation for developing models and computational pipelines in PLF, a more comprehensive understanding of the field’s challenges and opportunities will require further datasets encompassing diverse farming conditions and animal species.

3 Materials and Methods

Cow husbandry

All procedures involving cow handling and image capturing were conducted in accordance with ethical guidelines and approved by the Virginia Tech Institutional Animal Care and Use Committee (IACUC #22-146). The cows studied were

part of the dairy herd at the Virginia Tech Dairy Complex in Blacksburg, Virginia, USA, which comprises approximately 80% Holstein and 20% Jersey cows. For the 'External' setting, the study included 100% Holstein cows. The milking cows were housed in pens within a free-stall barn, featuring two rows of sand-bedded stalls, headlocks at the feed bunk, and two water troughs per pen. The stocking density was maintained at 100% (i.e., one cow per stall). Heat stress was managed using automatic 48-inch diameter fans positioned over the stalls and feeding alleys. Cows were milked twice daily at 1:00 am and 12:00 pm in a double-twelve parallel milking parlor. They were fed ad libitum (with less than 5% refusals) once daily at 8:00 am with a total mixed ration (TMR) consisting of approximately 42% corn silage, 8% grass hay, and 50% concentrate on a dry matter basis. Manure from the stalls was removed at each milking session by personnel driving the cows to milking. Manure from the walking alleys within the pen was cleared two or three times daily using an automatic flushing system with recycled water. Fresh or recycled sand was added on a weekly basis.

Study datasets

This study consists of two datasets: the COLO dataset and the external dataset collected from Inner Mongolia University for model evaluation [43]. The COLO dataset was collected using the Amazon Ring camera model Spotlight Cam Battery Pro (Ring Inc.), which offers a real-time video feed of dairy cows. Three cameras were installed in the barn: two at a height of 3.25 meters (10.66 feet) above the ground covering an area of 33.04 square meters (355.67 square feet). One camera provided a top view while the other was angled approximately 40 degrees from the horizontal to offer a side view of the cows. These are hereafter referred to as *the Top-View camera* and *the Side-View camera*, respectively. A third camera, termed *the external camera*, was set at a lower height of 2.74 meters (9.00 feet) and covered a larger area of 77.63 square meters (835.56 square feet). Positioned 10 degrees downward from the horizontal, it captured a challenging perspective prone to occlusions among cows. Since the images collected from the third camera have a different view angle and lighting conditions, they were used as an external dataset, named *ext_VT*, for model evaluation. The second dataset, named *ext_IMU*, was collected from Inner Mongolia University, China, using surveillance cameras (Dahua cameras, M/K model) to monitor the cow daily behaviors, such as feeding, drinking, and ruminating in the barn [43]. This study sampled 250 images with 3,699 cow instances to form the *ext_IMU* dataset, that has different farm layout and camera angles from the COLO dataset. The example images from the datasets are shown in **Figure 1**.

Images were captured using an Ring Application Programming Interface (API) [44], configured to record a ten-second video clip every 30 minutes continuously for 14 days. Since the image quality relies on the camera's internet connection, which was occasionally unstable, some images were found to be tearing or unrecognizable. Hence, the resulting dataset was manually curated for consistent quality, comprising 504 images from *the Top-View camera*, 500 from *the Side-View camera*, and 250 from *the external camera*. These images were further categorized based on the lighting conditions: for *the Top-View camera*, 296 images were captured during daylight, 118 in the evening under artificial lighting, and 90 as near-infrared images without artificial light. From *the Side-View camera*, 113 images were taken in the evening, and 97 as near-infrared images. All images from *the external camera* were captured during the day.

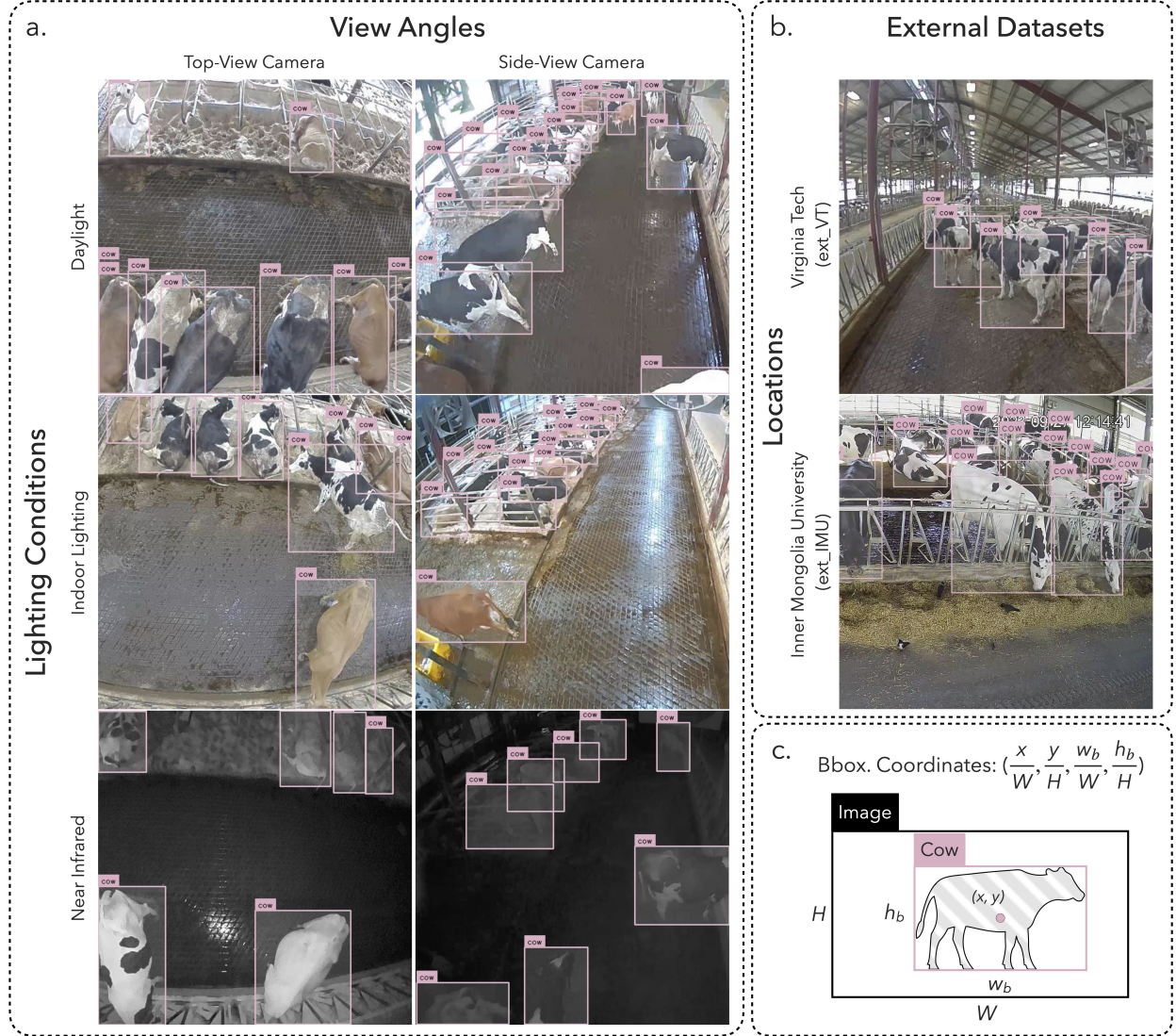


Figure 1: Overview of the COLO dataset and external dataset. (a) Six instance images from the dataset with pink bounding boxes labeling the location of cows. The columns show two different view angles: top-view and side-view. The rows show three different lighting conditions: daylight, indoor, and near-infrared. (b) Example images of the external datasets: ext_VT and ext_IMU. (c) A visual example of the annotated image in YOLO format that is used in this study. W , H , w_b , and h_b represent the width, height, width of the bounding box, and height of the bounding box, respectively. x and y represent the center coordinates of the bounding box.

- 1 The image annotations were conducted using an online platform, Roboflow [31], to define cow positions in the images.
- 2 The bounding boxes were manually drawn to enclose the cow contours, providing the coordinates of the top-left corners
- 3 and the width and height of the boxes. If cows were partially occluded, the invisible parts were inferred based on the
- 4 adjacent visible parts. If the cow position was too far from the camera, making important body features such as the head,
- 5 tail, and legs unrecognizable, the cow was excluded from the dataset. The final annotations were saved in the YOLO
- 6 format [45], where annotations were stored in a text file with one row per cow in the image, each row containing the

cow’s class, center coordinates, width, and height of the bounding box. The graphical representation of the annotated images is shown in **Figure 1c**.

Study 1: benchmarking model generalization across different environmental conditions

This study aimed to examine the first hypothesis that the model performance is equally affected by the view angle and lighting conditions. To compare the performance difference between view angles and lighting conditions, we designed a Monte Carlos cross-validation strategy where models were trained on one data split (e.g., top-view angle) with sampled images, and evaluated on a fixed set of images from another split (e.g., side-view angle). There are four training configurations in this study (**Figure 2a**):

- *Baseline*: The model was trained and evaluated on the dataset characterized for all conditions, including top-view, side-view, daylight, evening, and near-infrared images. The images did not overlap between the training and test sets.
- *Top2Side*: The model was trained on the top-view images and evaluated on the side-view images.
- *Side2Top*: The model was trained on the side-view images and evaluated on the top-view images.
- *Day2Night*: The model was trained on the daylight images and evaluated on the evening images, including both artificial lighting and near-infrared images.

To study how the training sample size affects model performance in each configuration, the testing set in the cross-validation was fixed to the same 100 images. Then, the training set size was iteratively altered from 16 to 500 images, with the sample size doubled at each step until reaching 500 images. Each training sample size was repeatedly sampled 20 times with different random seeds to ensure the robustness of the results.

To assess significant differences in mAP@0.5:0.95 performance among the four configurations, pairwise statistical tests were conducted. For each pair of configurations, data normality was first evaluated using the Shapiro-Wilk test [46]. If both were normal, homogeneity of variances was checked with Levene’s test [47]. Based on these tests, either a two-sample t-test (normal, equal variance) or Welch’s t-test [48] (normal, unequal variance) was used to compare means. If the data were not normally distributed, the Mann-Whitney U test [49] was applied to compare the distributions. The significance level was set at $p < 0.05$.

Study 2: the correlation between model complexity and performance on the tasks of localizing cows

To examine whether model performance improves with increasing complexity, three object detection model families were selected for this study: YOLOv11, YOLOv12, and RT-DETR. Within each family, two size variants were chosen to assess the impact of model complexity under a consistent architecture and mechanism. The selected models and their parameter counts (in parentheses) are as follows: YOLOv11n (2.6 million) and YOLOv11x (56.9 million), YOLOv12n (2.6 million) and YOLOv12x (59.1 million), and RT-DETR-l (45 million) and RT-DETR-x (86 million).

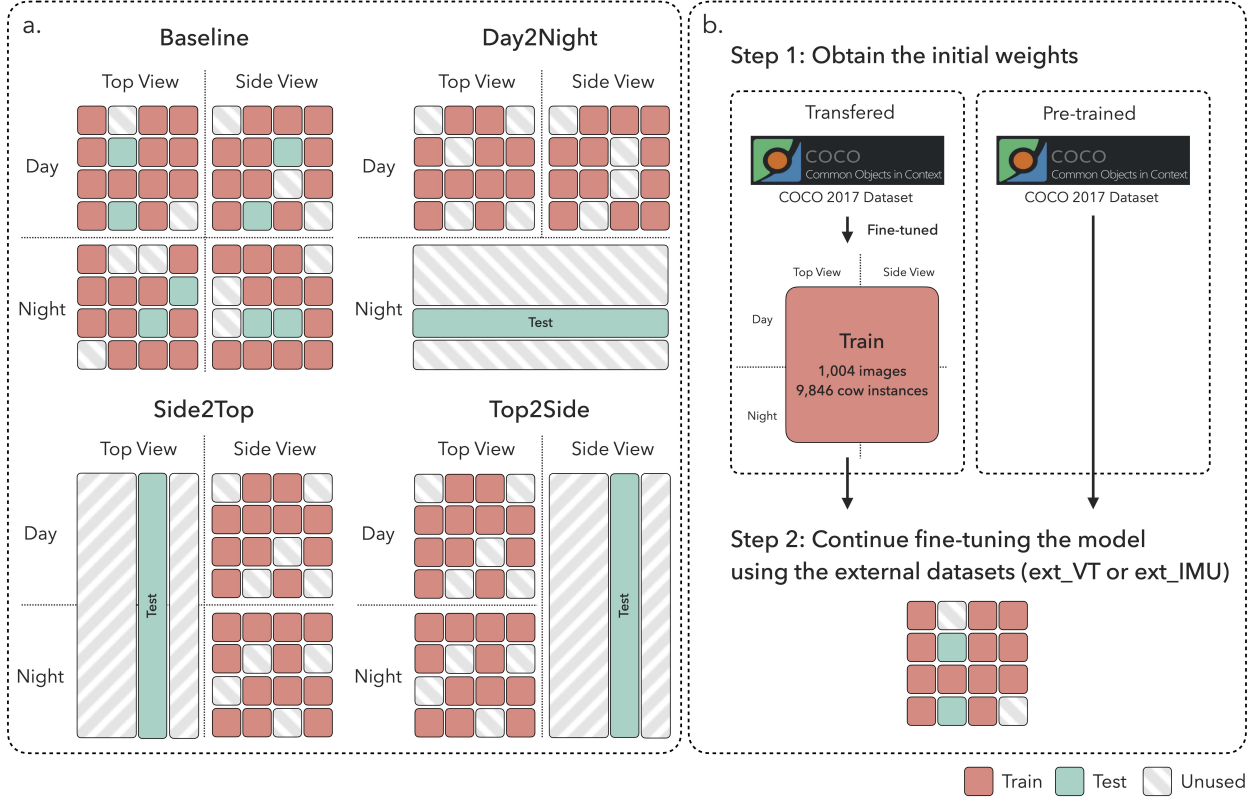


Figure 2: Experiment design for the model generalization study. (a) The experiments in Studies 1 and 2 contain four cross-validation configurations to investigate the effect of different environmental conditions on model performance: Baseline, Top2Side, Side2Top, and Day2Night. (b) The study 3 compares the performance of two different initial weights for fine-tuning the model on external datasets. Noted that the size of each rectangular box is not proportional to the number of images in the dataset. It is to visually show where the images are sampled from.

1 All models were evaluated using the same cross-validation configuration as in Study 1 (**Figure 2a**). For each model pair
 2 that consists of a smaller and a larger variant from the same family, performance differences were statistically tested
 3 as the same procedures as in Study 1. A significant difference between the two sizes within a family would indicate
 4 that model performance is influenced by complexity. Conversely, a lack of significant difference would suggest that
 5 comparable performance can be achieved with the smaller, more computationally efficient model.

6 Study 3: assessing the advantages of using a fine-tuned model over the pre-trained model as initial model weights

7 Most models are released with pre-trained weights obtained from large datasets containing millions of object instances
 8 (e.g., COCO [24] and ImageNet [34]). The pre-trained models have a general capability in recognizing common objects
 9 such as vehicles, animals, and household items. When the model is required to recognize specific objects (i.e., cows in
 10 this study), a model trained on a smaller but specific dataset is expected to have better performance. However, such
 11 advantages may not necessarily persist as the training sample size increases. Having an equally large number of samples
 12 for both the pre-trained and fine-tuned models could diminish the performance gap between the two models.

To investigate this hypothesis, we evaluated the performance of fine-tuned models on external datasets using two different initial weight configurations (**Figure 2b**). To compare the impact of different initializations, we used two sets of initial weights. The first was obtained by fine-tuning a pre-trained model on the COLO dataset presented in this work (1,004 images, 9,846 cow instances), while the second came directly from a model pre-trained on the COCO dataset. With these two sets of weights, we fine-tuned models on the external datasets, *ext_VT* and *ext_IMU*, using sample sizes of 16, 128, and 200 images for fine-tuning. Evaluation was performed on a fixed set of 50 unseen images from the same external dataset source. Similar to Study 1, the training set was sampled 20 times with different random seeds to account for sampling variability. This allowed us to track the gradual performance change as more images from the new environment were incorporated. Model performance was assessed using the mAP@0.5:0.95 metric. We conducted a pairwise comparison between the two initializations for each sample size and model variant using the same statistical procedures from Study 1 and Study 2. A statistically significant difference would suggest that transferring weights from a task-specific dataset (COLO) offers a greater advantage than using general-purpose pre-trained weights from a large dataset (COCO).

Model training

The model training was conducted using the Ultralytics framework [50], which provides predefined data augmentation strategies and optimized hyperparameters for the training process. Data augmentation aims to increase the diversity of the training dataset and improve the model robustness under varying conditions. For instance, horizontal flipping randomly mirrors images, helping prevent overfitting to specific orientations present in the training data. There are also other augmentation techniques that shift the color space of the images, mimicking the variations in lighting conditions. All these techniques are designed to enhance the model to better generalize to new environments that may differ from the training data. This study focused on data augmentation techniques that operate on a single image at a time, in contrast to more complex methods like CutMix [51] and MixUp [52] which combine multiple images in random, overlapping positions. This focus was maintained to minimize potential bias in evaluation introduced by the interaction of multiple images. **Figure 3** shows two instance images with different applied augmentations. The complete list of data augmentation techniques used in this study is detailed in **Table 1**.

Table 1: Data augmentation parameters of Ultralytics used in the study. The column of probability indicates the likelihood of applying the augmentation.

Augmentation Parameter	Description	Probability
flip_lr	Randomly flips the images horizontally to create a mirror image.	0.5
scale	Randomly scales the images to simulate different distances between the camera and the cows	0.5
translate	Randomly move the images to simulate different camera angles.	0.1
hsv_h	Randomly changes the hue of the images.	0.015
hsv_s	Randomly changes the saturation of the images.	0.7
hsv_v	Randomly changes the brightness of the images.	0.4

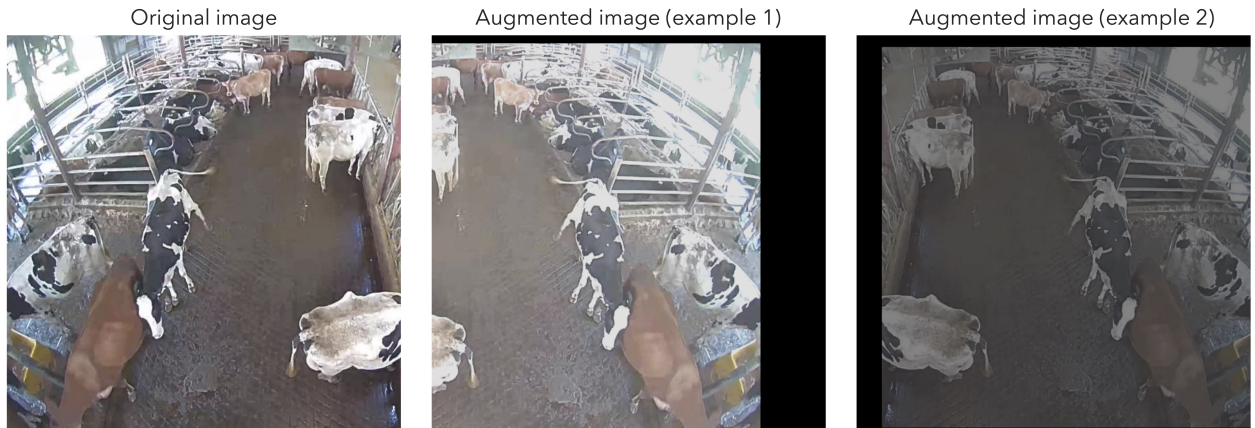


Figure 3: Two example images with different data augmentation applied. The left image is the original image, and the middle and right images are the augmented images. The augmentations include horizontal flipping, scaling, translation, and color space changes.

1 Additionally, the training process was governed by predefined hyperparameters, including the number of epochs, batch
2 size, optimizer, and learning rate. Rather than using the number of epochs as a fixed standard, this study adopted
3 a constant number of update steps set to 8,192 (i.e., 2^{13}). It ensured that all experiments, regardless of dataset size,
4 underwent the same amount of model updates. For example, if the sample size is 500, the number of epochs is $8192 /$
5 $(500 / 16) = 262.4 \approx 262$ epochs. This approach is especially important for experiments with smaller sample sizes,
6 where a fixed number of epochs would otherwise result in significantly fewer updates. By standardizing on update steps,
7 smaller datasets are trained over more epochs to reach the same number of updates, ensuring a fairer comparison across
8 experiments. Early stopping was applied with a patience of one-quarter of the total number of epochs, and training
9 was terminated if the model’s performance, measured by $\text{mAP}@0.5:0.95$, did not improve within this window. The

1 optimization process was performed using the Adam optimizer [53]. A detailed list of training hyperparameters is
 2 presented in **Table 2**.

Table 2: Hyperparameters for Ultralytics model training.

Hyperparameter	Description	Value
step	Number of update steps for training	8192
epochs	Number of time to iterate through the entire dataset	$8192 / (n / 16)^a$
batch	Number of images in each batch	16
optimizer	Optimizer used for training	Adam
lr0	Initial learning rate	0.01
lrf	Learning rate final value	0.1
momentum	Momentum for the optimizer	0.9

^a The sample size n is the number of images in the training dataset. The number of epochs is calculated as the total number of update steps divided the sample size divided by the batch size.

3 Model evaluation

4 The examined YOLO models are object detection models that return positions of detected objects (i.e., cows in this
 5 study) for the evaluated images. The detections are represented by a list of bounding boxes. Regardless of specific
 6 procedures among YOLO variants for computational efficiency, such as YOLOv8, which integrates objectness scores
 7 and conditional class probabilities into a single confidence score, each detection generally consists of $4 + c$ elements:
 8 the xy-coordinates, width, and height of the bounding box, and the c confidence scores indicating the probability of
 9 the object belonging to each of the c classes. The class with the highest confidence score is considered the predicted
 10 class of the object. To evaluate the model performance, two aspects are considered: the localization accuracy and
 11 the classification accuracy. The localization accuracy is measured by the Intersection over Union (IoU) between the
 12 predicted bounding box and the ground truth bounding box. On the other hand, the classification accuracy is measured
 13 by the precision and recall given the confidence threshold. If the confidence score of a detection is higher than the
 14 threshold, the detection is considered a positive detection. Otherwise, the detection is neglected. Mean Average
 15 Precision (mAP) is a key metric in object detection that combines both localization and classification accuracy. It is
 16 calculated by averaging the area under the precision-recall curve across all classes. This curve is generated by varying
 17 the confidence threshold from 0 to 1 for a given IoU threshold. In this study, comprehensive model performance was
 18 primarily evaluated using the averaged mAP across IoU thresholds ranging from 0.5 to 0.95, denoted as mAP@0.5:0.95.
 19 To provide a more intuitive understanding, we also report mAP at a less stringent IoU threshold of 0.5 (mAP@0.5),
 20 along with precision and recall at a confidence threshold of 0.25. The 0.25 threshold was selected as it is the default
 21 in the Ultralytics framework [50]. This threshold tends to yield higher recall, which is favorable in scenarios where
 22 missing detections are more critical than false positives.

1 Additionally, this study examined the computational cost of different models in terms of both processing time and
 2 memory usage during training and inference. Understanding computation time is crucial for models intended for
 3 real-time applications, which typically require a minimum of 30 frames per second (FPS) to meet real-time performance
 4 standards. Memory usage is another critical factor that affects the feasibility of deploying a model on specific hardware.
 5 For instance, training a large model that demands 30 gigabytes (GB) of video memory (VRAM) would exceed the
 6 capacity of a consumer-grade GPU like the NVIDIA 3080 Ti, which offers only 12 GB of VRAM.

7 For the training phase, we measured the wall clock time per update step, which is defined as the time required to
 8 process a batch of images and update model weights, along with the peak VRAM usage based on the log files generated
 9 by the TensorBoard [54]. During inference, FPS was measured by processing a batch of 16 images and recording
 10 the total time taken. The VRAM usage was also monitored during inference through PyTorch’s built-in function
 11 `torch.cuda.max_memory_allocated()` [55]. To ensure robustness, both training and inference measurements were
 12 averaged over 30 iterations. Training and inference were conducted on NVIDIA A100 80GB and NVIDIA L40S 48GB
 13 GPUs, while inference was additionally tested on NVIDIA 3080 Ti 12GB and NVIDIA 4090 24GB GPUs.

14 Software environment and hardware details

15 The computing environment for this study was configured across three different platforms. As multiple models were
 16 evaluated under varying hardware and software settings, details such as software versions, GPU specifications, and
 17 other system configurations are summarized in **Table 3** to clearly document the differences among the computing
 18 environments.

Table 3: System configuration summary across three GPU setups

Category	System 1 (A100)	System 2 (L40S)	System 3 (3080Ti / 4090)
GPU	NVIDIA A100-SXM4 80GB	NVIDIA L40S 48GB	NVIDIA 4090 24GB NVIDIA 3080 Ti 12GB
CPU / System	HPE Apollo 6500	Dell PowerEdge R760XA	ASUS X670E + Ryzen 9 7950X
Operation System	Rocky Linux 9.5	Rocky Linux 9.4	Ubuntu 24.04.2
Python Version	3.11.5	3.11.5	3.11.8
PyTorch Version	2.4.0+cu121	2.4.0+cu121	2.3.1+cu121
NVIDIA Driver	550.127.05	550.54.15	550.144.03
CUDA Version	12.4	12.4	12.4
cuDNN Version	9.0.1	9.0.1	8.9.2
Ultralytics Version	8.3.89	8.3.89	8.3.96

4 Results and Discussion

Public dataset: COLO

The COLO dataset contains 1,254 images and 11,818 cow instances captured from an indoor farm environment. The dataset is organized in YOLO and COCO formats and published on the online platforms GitHub (<https://github.com/Niche-Squad/COLO/>). The dataset consists of eight configurations (**Table 4**): *0_all*, *1_top*, *2_side*, *a1_t2s*, *a2_s2t*, *b_light*, *ext_VT*, and *ext_IMU*. The *0_all* configuration serves as the baseline for this study, featuring non-overlapping training and testing images collected from both the Top-View Camera and Side-View Camera. The *1_top*, *2_side*, and *ext_VT* configurations contain images from their respective cameras. The *a1_t2s*, *a2_s2t*, and *b_light* configurations include training/testing splits for the *Top2Side*, *Side2Top*, and *Day2Night* scenarios, respectively. The *ext_VT* and *ext_IMU* configuration is used to evaluate the model’s performance on unseen data. Both training and testing images in these configurations are collected from the same camera, but the images are different. The *ext_VT* configuration uses images from the Virginia Tech dataset, while the *ext_IMU* configuration uses images gathered at Inner Mongolia University [43].

Table 4: Overview of the COLO dataset configurations.

Experiment	Configuration (folder name)	Training Source	Testing Source
Study 1 and Study 2	<i>Baseline (0_all)</i>	Top-View + Side-View	Top-View + Side-View
	<i>Top2Side (a1_t2s)</i>	Top-View	Side-View
	<i>Side2Top (a2_s2t)</i>	Side-View	Top-View
	<i>Day2Night (b_light)</i>	Day	Night
Study 3	<i>ext_VT (ext_VT)</i>	Virginia Tech	Different images, same source
	<i>ext_IMU (ext_IMU)</i>	Inner Mongolia University	Different images, same source
Unused	<i>1_top</i>	Top-View	Top-View
	<i>2_side</i>	Side-View	Side-View

Study 1: the changes in camera view angles dramatically affect model performance

Regardless of model architecture, the evaluated performance across the four configurations exhibits a consistent trend: the *Baseline* configuration consistently outperforms the other configurations, then *Day2Night*, followed by *Side2Top*, and finally *Top2Side*. All the outperformance differences are statistically significant. It is found that changing the camera view from top-view to side-view results in a substantial performance drop. Using the *Baseline* configuration as a reference and evaluating with the mAP@0.5:0.95 metric on 500 training samples, the performance decreased by 51.6%, 50.5%, and 44.1% for YOLOv11, YOLOv12, and RT-DETR, respectively. The second most significant drop was observed when switching the camera view from side-view to top-view, with performance declines of 31.4%, 30.5%,

and 16.5% for YOLOv11, YOLOv12, and RT-DETR, respectively. Finally, altering lighting conditions from day to night also led to a noticeable drop—approximately 10%, though this effect was less severe compared to changes in camera view angle (**Figure 4**).

Another noteworthy observation is that the trend of improved performance with increasing training sample size is not consistent across all model architectures. For instance, in the *Day2Night* configuration, both YOLOv11 and YOLOv12 showed smaller performance gains when increasing the training set from 128 to 500 samples compared to the gain from 32 to 128 samples. This suggests diminishing returns and saturated performance gains with larger training sizes. In contrast, the RT-DETR model demonstrated greater marginal improvements from 128 to 500 samples across the *Day2Night*, *Side2Top*, and *Top2Side* configurations than from 32 to 128 samples. Although this observation is based on limited data and context, it suggests that RT-DETR is more data-hungry than YOLOv11 and YOLOv12. This aligns with the findings of [56], which report that DETR-based models generally require more data than CNN-based models like YOLO.

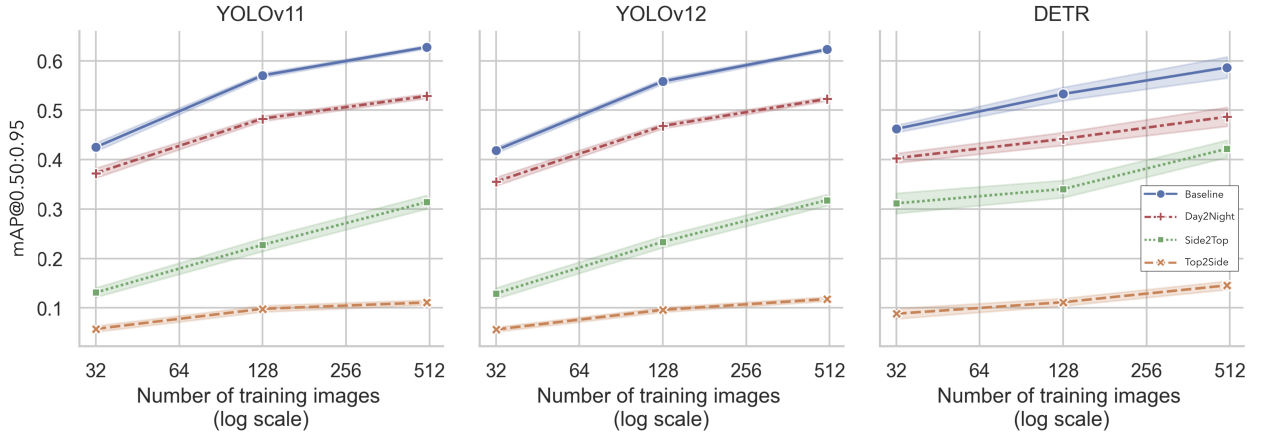


Figure 4: The generalization performance of the three model architectures, including YOLOv11, YOLOv12, and RT-DETR, is evaluated across various configurations. For each architecture, performance is averaged over its two size variants. Evaluation is based on the metric mAP@0.5:0.95 using 32, 128, and 500 training samples. Different configurations are distinguished by unique color and marker combinations. The shaded area around each line indicates two standard deviations.

Despite the various challenges in adapting models from day to night conditions, the *Day2Night* configuration consistently maintains higher mAP@0.5:0.95, closely mirroring the *Baseline* configuration across all training sample sizes and model architectures. This suggests that changes in lighting have less impact on the model’s ability to detect objects compared to changes in viewpoint. This robustness to lighting could be attributed to the inclusion of diverse lighting conditions in the training phase. Specifically, model performance benefited from pixel-wise augmentation techniques such as adjustments to hue, saturation, and value (HSV). These augmentations introduced a variety of color variations to the images, enhancing the model’s ability to generalize across different visual conditions. Moreover, these object detection models benefit from pre-training on the COCO dataset, which is characterized by a wide array of images with varied lighting, aiding their adaptability to shifts in light.

On the other hand, the models perform suboptimally in scenarios involving changes in viewpoint. Each new viewpoint introduces fundamentally different object features that are not replicated through standard data augmentation methods such as lighting or affine transformations. For example, when the camera is placed at a lower angle, cows are more frequently occluded by stalls and fences. These additional objects introduce variations that cannot be mitigated by augmentations in HSV space or image translation. Consequently, *Top2Side* performs the worst, as it is particularly challenging to identify cows from the side. In summary, camera view angle is crucial for model generalization, with side views being the most challenging.

Study 2: a higher model complexity does not always lead to better performance

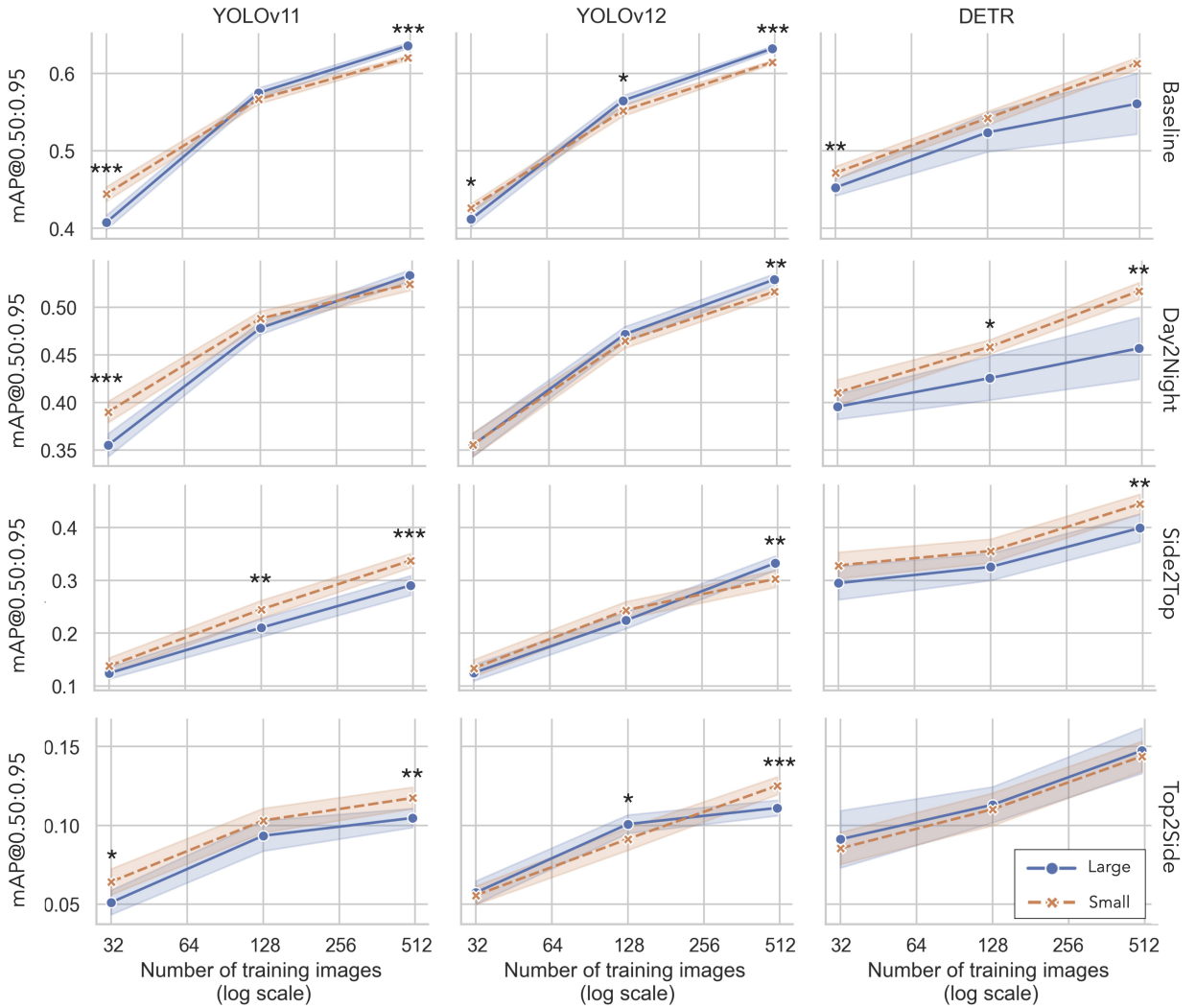


Figure 5: A detailed breakdown of performance by size variants within each model architecture is presented. Configurations are arranged along the rows, while columns represent the different model architectures. Statistical significance of the performance differences between the two size variants is denoted using asterisks, with the following legend: * = $p < 0.05$, ** = $p < 0.01$, and *** = $p < 0.001$. The solid blue line represents the large model variant, while the dashed orange line represents the small model variant. The shaded area around each line indicates two standard deviations.

The study found that training configuration significantly influences the relationship between model complexity and performance. The subset results for YOLOv11 are summarized in **Table 5**. The results for YOLOv12 and RT-DETR can be found in the supplementary material. Challenging tasks, such as predicting images across different camera angles (e.g., *Top2Side* and *Side2Top*) as suggested by Study 1, revealed that smaller model variants can sometimes outperform their larger counterparts. Specifically, when training with 500 image samples, smaller models achieved better performance in four out of six comparisons across the three model architectures and two camera-angle configurations. Conversely, in simpler tasks like the *Baseline* and *Day2Night* configurations, the trend reversed, with larger models generally outperforming smaller ones, except for DETR models. The most significant outperformance by a larger model occurred in the *Baseline* configuration, where YOLOv12x surpassed YOLOv12n by 1.7%, and YOLOv11x outperformed YOLOv11n by 1.6%. DETR models, however, consistently performed better with their smaller variants, notably with the small RT-DETR variant showing a 5.2% increase in mAP@0.5:0.95.

These findings suggest that the relationship between model size and performance is not linear or universally consistent. Instead, it depends on task complexity and dataset characteristics. In more complex configurations, simpler models occasionally performed better, challenging the assumption that larger models are always superior. As illustrated in **Figure 5**, while all studied architectures (YOLOv11, YOLOv12, and RT-DETR) generally benefit from increased parameters when trained on large-scale datasets like COCO [24], this pattern does not hold in the current setting. The COCO dataset includes 80 object classes and diverse standalone images, whereas this study used a domain-specific indoor farm dataset focusing on a single class: cows. Under such narrow and homogenous conditions, larger models may not offer proportional gains, and model generalization behavior differs.

For example, the small YOLOv11n model (2.6M parameters) achieved 44.4% mAP@0.5:0.95 with just 32 training samples, outperforming a much larger model (52.1M parameters) that achieved only 40.8%. This advantage was especially pronounced in the *Top2Side* and *Side2Top* configurations, where YOLOv11n outperformed YOLOv11x by 1.2% and 4.8%, respectively, with 500 training samples.

These results underscore the value of deploying smaller models in targeted, homogeneous tasks such as cow positioning, where they offer an optimal balance between computational efficiency and prediction accuracy. They also reinforce the broader point: model selection should be tailored to the specific dataset and task, rather than relying on model complexity as a proxy for better performance. In summary, higher model complexity does not inherently guarantee improved outcomes. Instead, careful consideration of the task context and dataset is essential in selecting the most effective model architecture.

Table 5: Summary of YOLOv11 results for all configurations between the two model size variants. The results are represented as mean $\pm$ standard deviation. The greater mean value for each pair of model size variants is highlighted in bold.

Sample Size	Configuration	Model Variant	mAP@0.5:0.95	mAP@0.5	Precision	Recall
32	Baseline	YOLOv11x (56.9M)	0.408 $\pm$ 0.022	0.716 $\pm$ 0.025	0.807 $\pm$ 0.031	0.625 $\pm$ 0.032
		YOLO11n (2.6M)	0.444 $\pm$ 0.021	0.760 $\pm$ 0.017	0.822 $\pm$ 0.024	0.662 $\pm$ 0.023
	Day2Night	YOLOv11x (56.9M)	0.355 $\pm$ 0.027	0.624 $\pm$ 0.037	0.796 $\pm$ 0.038	0.539 $\pm$ 0.046
		YOLO11n (2.6M)	0.390 $\pm$ 0.025	0.673 $\pm$ 0.023	0.805 $\pm$ 0.032	0.582 $\pm$ 0.029
	Side2Top	YOLOv11x (56.9M)	0.124 $\pm$ 0.024	0.292 $\pm$ 0.063	0.473 $\pm$ 0.062	0.287 $\pm$ 0.094
		YOLO11n (2.6M)	0.138 $\pm$ 0.034	0.301 $\pm$ 0.068	0.484 $\pm$ 0.082	0.283 $\pm$ 0.069
	Top2Side	YOLOv11x (56.9M)	0.051 $\pm$ 0.017	0.121 $\pm$ 0.034	0.258 $\pm$ 0.084	0.126 $\pm$ 0.038
		YOLO11n (2.6M)	0.064 $\pm$ 0.018	0.147 $\pm$ 0.034	0.248 $\pm$ 0.100	0.131 $\pm$ 0.036
128	Baseline	YOLOv11x (56.9M)	0.575 $\pm$ 0.014	0.868 $\pm$ 0.010	0.899 $\pm$ 0.018	0.771 $\pm$ 0.019
		YOLO11n (2.6M)	0.567 $\pm$ 0.014	0.875 $\pm$ 0.011	0.896 $\pm$ 0.015	0.788 $\pm$ 0.014
	Day2Night	YOLOv11x (56.9M)	0.478 $\pm$ 0.015	0.738 $\pm$ 0.018	0.869 $\pm$ 0.021	0.638 $\pm$ 0.024
		YOLO11n (2.6M)	0.488 $\pm$ 0.017	0.772 $\pm$ 0.019	0.873 $\pm$ 0.020	0.687 $\pm$ 0.030
	Side2Top	YOLOv11x (56.9M)	0.210 $\pm$ 0.039	0.426 $\pm$ 0.064	0.607 $\pm$ 0.087	0.387 $\pm$ 0.055
		YOLO11n (2.6M)	0.245 $\pm$ 0.038	0.472 $\pm$ 0.063	0.651 $\pm$ 0.050	0.426 $\pm$ 0.056
	Top2Side	YOLOv11x (56.9M)	0.093 $\pm$ 0.022	0.186 $\pm$ 0.034	0.393 $\pm$ 0.072	0.164 $\pm$ 0.027
		YOLO11n (2.6M)	0.103 $\pm$ 0.017	0.207 $\pm$ 0.028	0.431 $\pm$ 0.082	0.181 $\pm$ 0.028
500	Baseline	YOLOv11x (56.9M)	0.636 $\pm$ 0.008	0.932 $\pm$ 0.005	0.919 $\pm$ 0.011	0.864 $\pm$ 0.011
		YOLO11n (2.6M)	0.620 $\pm$ 0.006	0.927 $\pm$ 0.005	0.910 $\pm$ 0.012	0.856 $\pm$ 0.012
	Day2Night	YOLOv11x (56.9M)	0.533 $\pm$ 0.013	0.817 $\pm$ 0.018	0.880 $\pm$ 0.028	0.726 $\pm$ 0.026
		YOLO11n (2.6M)	0.524 $\pm$ 0.015	0.820 $\pm$ 0.018	0.867 $\pm$ 0.023	0.733 $\pm$ 0.023
	Side2Top	YOLOv11x (56.9M)	0.290 $\pm$ 0.042	0.541 $\pm$ 0.056	0.663 $\pm$ 0.068	0.480 $\pm$ 0.049
		YOLO11n (2.6M)	0.338 $\pm$ 0.030	0.612 $\pm$ 0.043	0.728 $\pm$ 0.048	0.547 $\pm$ 0.040
	Top2Side	YOLOv11x (56.9M)	0.105 $\pm$ 0.014	0.192 $\pm$ 0.020	0.413 $\pm$ 0.051	0.193 $\pm$ 0.018
		YOLO11n (2.6M)	0.117 $\pm$ 0.015	0.222 $\pm$ 0.026	0.465 $\pm$ 0.069	0.210 $\pm$ 0.030

1 Study 3: the advantages of transferred initial weights are limited to similar domains

2 The benefits of transferring weights from a related task varied significantly across the two evaluated datasets (**Figure 6**).
3 On the more similar dataset, *ext_VT*, models initialized with transferred weights consistently outperformed those with
4 general pre-trained weights. These differences were statistically significant for most models, with the small YOLOv11
5 variant being the sole exception. While the performance gap narrowed as the number of training samples from the new
6 source increased, the advantage of transferred weights largely persisted even when training with the full 200-image
7 dataset. Each model architecture benefited from transferred weights in mAP@0.5:0.95, showing improvements of

1 2.62% (RT-DETR-x), 1.76% (YOLOv12-x), and 0.98% (YOLOv11-x) compared to using pre-trained weights. We also
 2 observed that larger model variants tended to benefit more from transferred weights; for example, the large RT-DETR
 3 model show a 2.62% advantage, whereas its small variant gained only 1.02%. Similar, though smaller, trends were
 4 noted for YOLOv12 (1.03%) and YOLOv11 (0.68%).

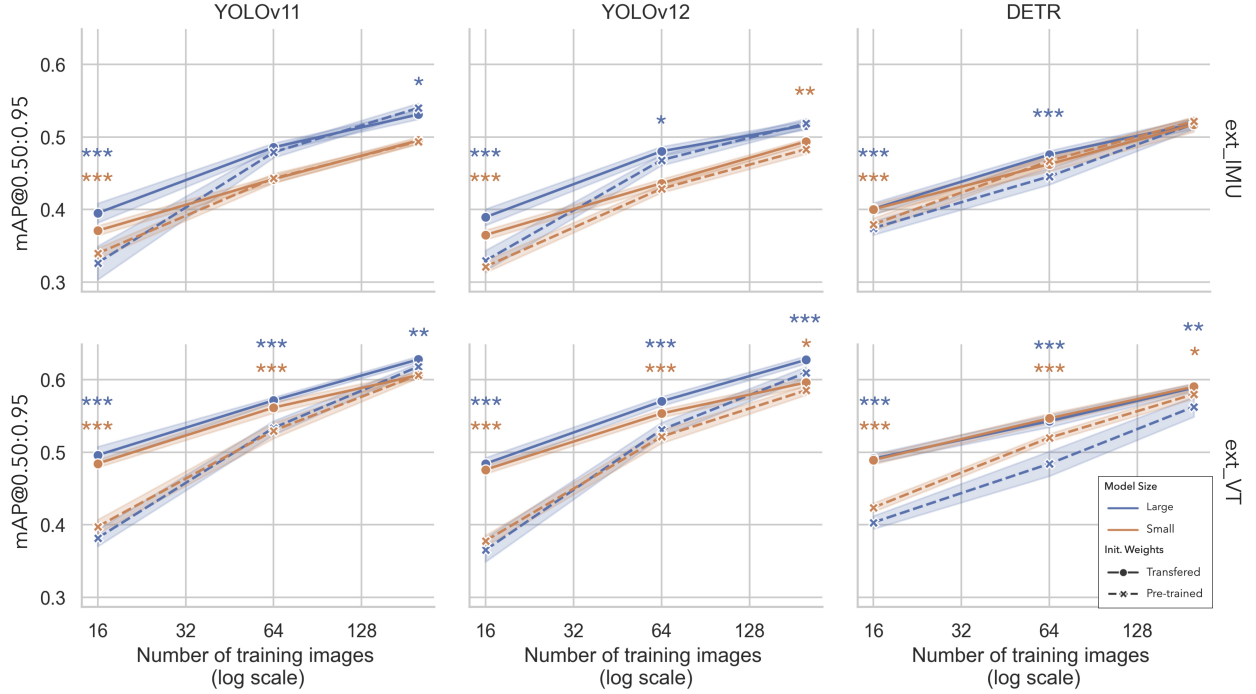


Figure 6: Fine-tuning performance (mAP@0.5:0.95) with different initial weights. Rows represent external datasets; columns represent model architectures. Solid and dashed lines indicate transferred and pre-trained weights, respectively. Blue and orange lines denote large and small model variants. Shaded areas show two standard deviations. Asterisks indicate t-test significance of performance differences between models initialized with pre-trained weights and those using transferred weights. Statistical significance is indicated by asterisks, following this legend : * = $p < 0.05$, ** = $p < 0.01$, *** = $p < 0.001$.

5 In contrast, for the more distinct *ext_IMU* dataset, which featured a considerably different barn setting, the advantage of
 6 transferred weights was only evident when training data was limited (e.g., 16 images). As the sample size increased to
 7 200, the performance difference between models fine-tuned with pre-trained and transferred weights largely disappeared
 8 across nearly all architectures. Notably, the YOLOv11x even showed a slight but significant advantage with pre-trained
 9 weights. These results strongly suggest that transferred weights offer greater benefits when the source and target
 10 domains are closely aligned. For more divergent domains, fine-tuning from general pre-trained weights may prove
 11 equally or more efficient. Furthermore, contrary to the findings in Study 2, which suggested that smaller models can
 12 sometimes outperform larger ones, this was not observed with the *ext_IMU* dataset. For YOLOv11 and YOLOv12,
 13 larger models consistently outperformed their smaller counterparts, particularly when transferred weights were used.
 14 This finding implies that larger models might be better equipped to leverage transferred weights in a new environment,
 15 as evidenced by performance gaps of 3.58% (YOLOv11) and 2.25% (YOLOv12) between model sizes with 200 training

1 images. The detailed performance of YOLOv11 is summarized in **Table 6**. The results for YOLOv12 and RT-DETR
2 can be found in the supplementary material.

3 Given these findings in this study, it is recommended that when the source and target domains are relatively similar (e.g.,
4 different areas within the same barn with comparable lighting and infrastructure), use transferred weights to initialize
5 the model. This approach can significantly boost model performance, especially when collecting and annotating new
6 data is resource-intensive. Conversely, when deploying the model in a distinctly different environment, even if it's
7 for detecting the same cow breeds (e.g., Holstein), it's advisable to train the model directly using general pre-trained
8 weights. The advantage of transferred weights diminishes as more data from the new environment becomes available,
9 and the performance gap between the two initialization approaches becomes negligible.

Table 6: Summary of the performance of YOLOv11 with different initial weights on the *ext_IMU* and *ext_VT* datasets. The results are represented as mean $\pm$ standard deviation. For each pair of initial weights, the greater mean value is highlighted in bold. Abbreviations: M (million), Config. (Configuration), Init. Weights (Initial Weights), mAP (Mean Average Precision). Note: If mean values are equal to two decimal places as shown in the table, the third decimal place is used to determine the greater value.

Sample Size	Config.	Model Size	Init. Weights	mAP@0.5:0.95	mAP@0.5	Precision	Recall
16	e_IMU	YOLOv11x (56.9M)	Transferred	0.40 $\pm$ 0.03	0.72 $\pm$ 0.04	0.77 $\pm$ 0.03	0.62 $\pm$ 0.04
			Pre-trained	0.33 $\pm$ 0.05	0.63 $\pm$ 0.08	0.67 $\pm$ 0.09	0.55 $\pm$ 0.05
		YOLO11n (2.6M)	Transferred	0.37 $\pm$ 0.02	0.72 $\pm$ 0.03	0.77 $\pm$ 0.04	0.62 $\pm$ 0.03
			Pre-trained	0.34 $\pm$ 0.03	0.68 $\pm$ 0.04	0.73 $\pm$ 0.05	0.59 $\pm$ 0.04
	e_VT	YOLOv11x (56.9M)	Transferred	0.50 $\pm$ 0.03	0.81 $\pm$ 0.03	0.83 $\pm$ 0.03	0.71 $\pm$ 0.04
			Pre-trained	0.38 $\pm$ 0.03	0.68 $\pm$ 0.03	0.76 $\pm$ 0.05	0.58 $\pm$ 0.06
		YOLOv11n (2.6M)	Transferred	0.48 $\pm$ 0.02	0.82 $\pm$ 0.02	0.82 $\pm$ 0.04	0.72 $\pm$ 0.03
			Pre-trained	0.40 $\pm$ 0.03	0.73 $\pm$ 0.03	0.78 $\pm$ 0.04	0.64 $\pm$ 0.04
64	e_IMU	YOLOv11x (56.9M)	Transferred	0.49 $\pm$ 0.01	0.81 $\pm$ 0.01	0.82 $\pm$ 0.02	0.71 $\pm$ 0.03
			Pre-trained	0.48 $\pm$ 0.02	0.79 $\pm$ 0.01	0.81 $\pm$ 0.03	0.70 $\pm$ 0.03
		YOLOv11n (2.6M)	Transferred	0.44 $\pm$ 0.01	0.79 $\pm$ 0.01	0.84 $\pm$ 0.02	0.69 $\pm$ 0.02
			Pre-trained	0.44 $\pm$ 0.01	0.78 $\pm$ 0.01	0.83 $\pm$ 0.03	0.67 $\pm$ 0.03
	e_VT	YOLOv11x (56.9M)	Transferred	0.57 $\pm$ 0.01	0.88 $\pm$ 0.02	0.87 $\pm$ 0.02	0.80 $\pm$ 0.02
			Pre-trained	0.53 $\pm$ 0.02	0.84 $\pm$ 0.01	0.86 $\pm$ 0.02	0.76 $\pm$ 0.02
		YOLOv11n (2.6M)	Transferred	0.56 $\pm$ 0.01	0.88 $\pm$ 0.01	0.87 $\pm$ 0.02	0.80 $\pm$ 0.02
			Pre-trained	0.53 $\pm$ 0.02	0.85 $\pm$ 0.01	0.87 $\pm$ 0.02	0.75 $\pm$ 0.02
200	e_IMU	YOLOv11x (56.9M)	Transferred	0.53 $\pm$ 0.01	0.85 $\pm$ 0.01	0.85 $\pm$ 0.03	0.77 $\pm$ 0.02
			Pre-trained	0.54 $\pm$ 0.01	0.86 $\pm$ 0.01	0.84 $\pm$ 0.02	0.78 $\pm$ 0.01
		YOLOv11n (2.6M)	Transferred	0.50 $\pm$ 0.01	0.85 $\pm$ 0.01	0.88 $\pm$ 0.02	0.76 $\pm$ 0.02
			Pre-trained	0.49 $\pm$ 0.01	0.84 $\pm$ 0.01	0.87 $\pm$ 0.02	0.74 $\pm$ 0.01
	e_VT	YOLOv11x (56.9M)	Transferred	0.63 $\pm$ 0.01	0.91 $\pm$ 0.01	0.89 $\pm$ 0.02	0.85 $\pm$ 0.01
			Pre-trained	0.62 $\pm$ 0.01	0.90 $\pm$ 0.01	0.89 $\pm$ 0.02	0.84 $\pm$ 0.02
		YOLOv11n (2.6M)	Transferred	0.61 $\pm$ 0.01	0.91 $\pm$ 0.01	0.90 $\pm$ 0.02	0.84 $\pm$ 0.02
			Pre-trained	0.61 $\pm$ 0.01	0.90 $\pm$ 0.01	0.89 $\pm$ 0.02	0.83 $\pm$ 0.02

1 Computational resource requirements

Evaluating computational resource requirements is essential for determining the feasibility of deploying object detection models in real-world scenarios, particularly in environments with limited hardware capabilities. This study examines the training time, inference performance, and memory usage of all evaluated models (**Figure 7, Table 7, Table 8**). Training time for each model was measured and expressed relative to a baseline, denoted as 1 $\times$. This baseline corresponds to the fastest observed training configuration: the YOLOv11n model running on an NVIDIA L40S GPU at 0.6 seconds

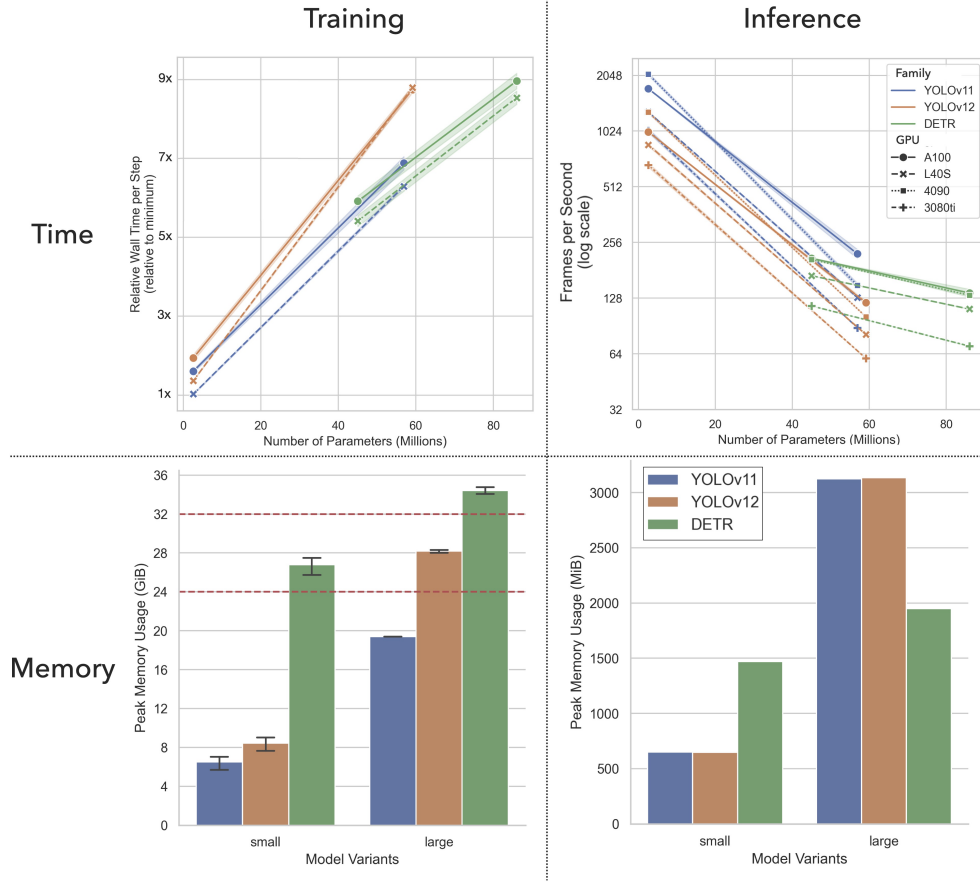


Figure 7: Comparative evaluation of computational resource requirements. (Top-left) Relative training time per step with 0.6 seconds as a baseline 1x. (Top-right) Inference time in frames per second for a batch of 16 images. (Bottom-left) GPU memory peak usage during training. (Bottom-right) GPU memory peak usage during inference. The red dashed lines in the bottom left panel indicates two common GPU memory sizes of consumer-grade GPUs.

per training step. Across all architectures on the same GPU, training time generally scaled linearly with the number of model parameters. An exception was observed with YOLOv12, which exhibited 29.5% and 37.6% longer training times compared to YOLOv11 on the A100 and L40S GPUs, respectively. This discrepancy is likely attributable to YOLOv12’s hybrid architecture, which integrates elements from both YOLOv11 and RT-DETR, resulting in additional computational complexity. All model-GPU combinations demonstrated inference speeds exceeding 60 FPS when processing a batch of 16 images. Although there is no universally accepted definition of real-time inference, a common benchmark is the frame rate of input devices, such as cameras. Human visual perception typically requires a minimum of 30 FPS for smooth real-time experiences. Thus, all models evaluated in this study meet or exceed this practical threshold for real-time deployment. GPU memory usage is another critical factor in determining the compatibility of models with specific hardware. High-end consumer-grade GPUs, such as the NVIDIA 5090 with 32 GB of memory, are capable of training all models in the study except RT-DETR-x, which demands an average of 34.3 GiB of memory. For more commonly available GPUs, such as the NVIDIA 4090 with 24 GB of VRAM or 3080ti with 12 GB, certain larger models, including the RT-DETR variants and YOLOv12x, exceed feasible memory limits. These elevated memory

Table 7: Training time and memory usage for each model (mean $\pm$ std).

Model size category	Model	Params. (M)	Time per step (sec.) A100 (80 GB)	Time per step (sec.) L40S (48 GB)	Memory usage (GiB)
Small	YOLOv11n	2.6	0.97 $\pm$ 0.03	0.62 $\pm$ 0.01	5.92 $\pm$ 1.08
	YOLOv12n	2.6	1.17 $\pm$ 0.06	0.82 $\pm$ 0.01	7.67 $\pm$ 1.21
	RT-DETR-l	45.0	3.56 $\pm$ 0.09	3.26 $\pm$ 0.10	24.46 $\pm$ 2.46
Large	YOLOv11x	56.9	4.14 $\pm$ 0.11	3.79 $\pm$ 0.03	18.08 $\pm$ 1.00
	YOLOv12x	59.1	5.26 $\pm$ 0.06	5.29 $\pm$ 0.04	26.89 $\pm$ 1.07
	RT-DETR-x	86.0	5.39 $\pm$ 0.13	5.14 $\pm$ 0.11	31.60 $\pm$ 3.18

Table 8: Inference frame per second (FPS) and memory usage for each model (mean $\pm$ std).

Model size category	Model	Params. (M)	FPS A100 (80 GB)	FPS L40S (48 GB)	FPS 4090 (24 GB)	FPS 3080Ti (12 GB)	Memory usage (MiB)
Small	YOLOv11n	2.6	1734.90 $\pm$ 14.71	1297.60 $\pm$ 2.25	2079.76 $\pm$ 86.72	1030.26 $\pm$ 67.80	650.00
	YOLOv12n	2.6	1013.95 $\pm$ 8.37	862.18 $\pm$ 36.41	1293.46 $\pm$ 46.96	670.14 $\pm$ 45.49	648.75
	RT-DETR-l	45.0	210.14 $\pm$ 0.43	168.98 $\pm$ 0.06	206.96 $\pm$ 1.62	116.14 $\pm$ 0.13	1469.92
Large	YOLOv11x	56.9	222.12 $\pm$ 33.46	129.08 $\pm$ 5.66	149.92 $\pm$ 11.13	88.38 $\pm$ 0.31	3125.37
	YOLOv12x	59.1	120.91 $\pm$ 6.97	81.30 $\pm$ 0.53	101.50 $\pm$ 1.23	60.48 $\pm$ 0.54	3133.62
	RT-DETR-x	86.0	136.31 $\pm$ 17.64	111.68 $\pm$ 0.44	132.64 $\pm$ 0.53	70.38 $\pm$ 0.40	1948.73

requirements can present financial and logistical challenges for both researchers and practitioners. However, these barriers can be mitigated by selecting appropriately sized models with efficient initialization strategies. For instance, smaller models fine-tuned on related datasets can offer strong baseline performance while remaining within hardware constraints, as suggested by the findings of the Study 3.

This evaluation underscores the trade-offs between model complexity and computational efficiency. Larger models, such as advanced YOLO and DETR-based variants, offer improved accuracy at the cost of increased training time and memory consumption. By understanding these trade-offs, practitioners can make informed decisions about model selection based on their resource availability and application-specific requirements. Ultimately, balancing performance with computational feasibility is key to the successful deployment of object detection models in real-world settings.

Metric interpretation

In this work, while the mAP@0.5:0.95 metric is the primary indicator for overall model performance, other metrics such as mAP@0.5, precision, and recall are also reported to offer more practical insights into model behavior. The mAP@0.5:0.95 provides a comprehensive evaluation of a model’s prowess in both localization and classification tasks. In contrast, mAP@0.5 is a less stringent metric, more tolerant of localization inaccuracies, making it more practical for scenarios focused on counting objects rather than pinpointing their exact locations. It still offers a general measure of

performance by averaging metrics across all confidence thresholds, which serve as criteria for classifying predictions as positive or negative. Higher confidence thresholds tend to yield more accurate predictions but also increase the likelihood of false negatives, while lower thresholds can lead to more false positives. Therefore, to translate performance into more practical terms, such as determining how many cows are detected when ten are present or how many detected cows are actually cows, precision and recall with a predefined confidence threshold become more informative. For instance, if a model has a precision of 0.875 and a recall of 0.7, and there are 10 cows in an image, the model is expected to detect 8 cows. Of these, 7 would be true positives and 1 would be a false positive, meaning 3 cows in the image were missed. This indicates the model successfully detects 70% of the cows present but misclassifies 12.5% of its detections as false positives (i.e., actually not cows).

The relationships among these metrics are visually represented in **Figure 8**. The scatter plots illustrate the pairwise correlations among the four metrics, with each data point representing a single model evaluation iteration across different model size variants, training sample sizes, and data configurations. The studies reveal that $\text{mAP}@0.5:0.95$ is the least saturated among the metrics. When $\text{mAP}@0.5:0.95$ exceeds 0.6, $\text{mAP}@0.5$ is generally above 0.9, and its marginal gain is smaller than a one-unit change in $\text{mAP}@0.5:0.95$. Similar trends are observed for precision, which tends to cap around 0.9 when $\text{mAP}@0.5:0.95$ is greater than 0.4. Recall, however, shows less saturation than $\text{mAP}@0.5$ and precision, and it is more sensitive to changes in $\text{mAP}@0.5:0.95$.

This understanding allows for a more practical interpretation of the performance gaps observed in the studies of this work. For example, when comparing the effect of model complexity in Study 2, is a 1.6% outperformance in $\text{mAP}@0.5:0.95$ by a larger model variant, despite being statistically significant, truly meaningful when both models already achieve an $\text{mAP}@0.5:0.95$ greater than 0.6? In practical terms, this might translate to less than one misclassified cow per 100 cows with the smaller model. Such practical interpretations can be extended to other studies, revealing that the advantages of larger models or transferred weights may become negligible when evaluating models using these more practical metrics like precision and recall. Ultimately, while $\text{mAP}@0.5:0.95$ provides a robust technical benchmark, considering a broader range of metrics offers a more application-oriented understanding of model performance. This holistic view is crucial for deploying models effectively in real-world scenarios where specific operational requirements often dictate which performance aspects are most critical.

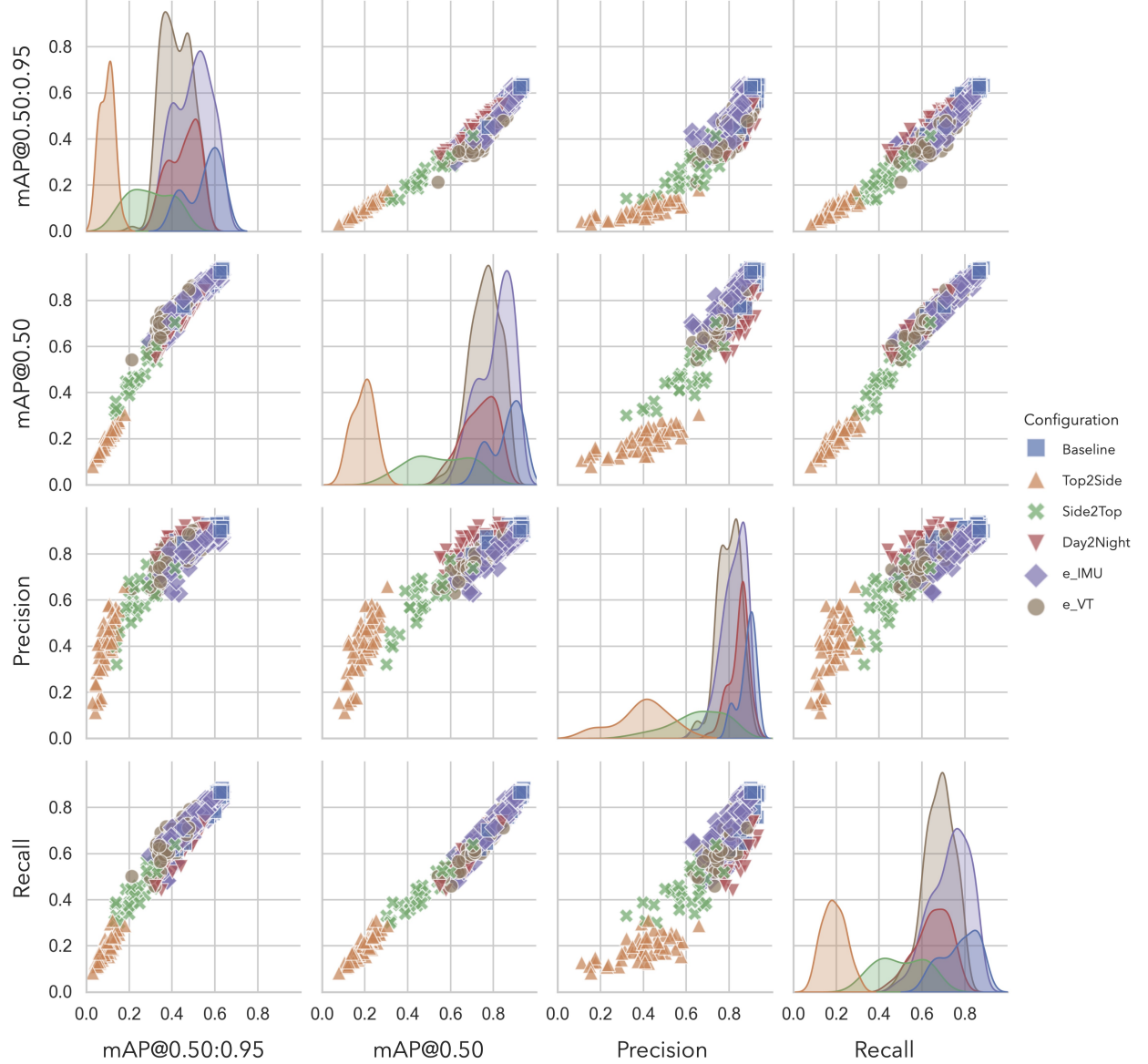


Figure 8: Pair-wise scatter plots illustrating the relationships among the four metrics used in this study. The metrics include mAP@0.5:0.95, mAP@0.5, precision, and recall. The diagonal panels show the distribution of each individual metric, while the off-diagonal panels display the correlation between each pair of metrics. In the scatter plots, each data point represents a single model evaluation iteration for a specific model size variant and data configuration. Different data configurations are distinguished by color. Only 500 iterations are shown for clarity.

1 Study limitations

While this study provides multi-aspects insights into single-object detection generalization using cows as a case study, several limitations warrant consideration when interpreting the results. Firstly, the focus on a single object class, cows, inherently restricts the direct generalizability of the findings to other object categories or more complex, multi-class detection scenarios. Cows possess specific visual characteristics and appear in relatively predictable environments

compared to the diversity encountered in broader object detection tasks. Consequently, the conclusion that camera angle changes pose a greater challenge than lighting variations might be specific to this context and may not hold true for objects with different textures, shapes, or typical backgrounds. Furthermore, while lighting and camera angles were investigated, other potentially significant environmental variables, such as partial occlusion (e.g., by fences, vegetation, or other animals), varying weather conditions, diverse background clutter, and significant intra-class variation (e.g., different breeds, ages, or postures of cows), were not the primary focus of the cross-validation strategies. The specific datasets used for training and validation, including their size and the diversity of conditions captured, could also influence the observed model performance and generalization capabilities.

Secondly, the investigation into model size and transfer learning, while informative, has its own constraints. The study compared a selection of model sizes and architectures; however, the findings might be dependent on the specific models chosen, and other architectures could potentially yield different results regarding robustness to the tested variations. The exploration of transfer learning demonstrated potential benefits for a similar task, but the effectiveness of transfer learning is highly contingent on the relationship between the source domain (i.e., where weights were initially trained) and the target domain (i.e., external barns), as well as the precise nature and degree of similarity of the "similar task" investigated. The performance gains observed might not be representative of transfer learning efficacy across tasks with varying degrees of relatedness or when using weights pre-trained on vastly different datasets. Additionally, the cross-validation strategies employed, while designed to isolate specific factors, represent only a subset of possible data splits and environmental challenges, and alternative validation approaches could reveal different facets of model generalization. Future work could expand the scope to include more object classes, a wider array of environmental variations and occlusions, diverse model architectures, and a more extensive analysis of transfer learning across different source and target task combinations to build upon the findings presented here.

5 Conclusion

This study investigated how various training configurations and model complexities affect the performance of object detection models designed for cow detection in indoor farm environments. Our findings highlight that model performance is highly dependent on camera viewpoints, with side views posing the most significant challenges. We also found that fine-tuning models with weights from similar datasets substantially improves performance, especially for complex models operating with limited data. To further support the research community, we've introduced COLO, a new public dataset for cow localization.

Our results indicate that while increasing model complexity can enhance performance, this isn't universally true, particularly in challenging scenarios, such as training models on top-view images and using them to predict side-view images. Models trained on a single viewpoint demonstrated limited generalization, emphasizing the critical need for diverse and consistent camera angles within training data.

Furthermore, while transfer learning from similar datasets can boost model performance when target datasets are small, it's crucial to ensure the training data accurately represents the target environment. The benefits of transfer learning diminish considerably when the new deployment environment considerably differs from the original training data. We also observed that larger models tend to leverage transfer learning more effectively than smaller models, likely due to their greater capacity to learn complex features.

Despite these valuable insights, our study has certain limitations. The models' performance was evaluated under specific indoor farm conditions, meaning the findings may not directly generalize to all livestock environments. Additionally, our reliance on predefined configurations might limit the applicability of our results to more dynamic settings. Future research should focus on developing adaptive methods to enhance model generalization across various viewpoints and environmental conditions. Specifically, incorporating advanced data augmentation techniques that simulate diverse camera angles in 3D space could effectively address performance degradation caused by different viewpoints.

In conclusion, this study offers practical guidance for reproducing model performance in new environmental settings and contributes the public COLO dataset to foster further research and advancements in this field.

Acknowledgments

This research was supported by the USDA Hatch Research Project funding VA-160196. The authors acknowledge Advanced Research Computing at Virginia Tech for providing computational resources and technical support that have contributed to the results reported within this paper. URL: <https://arc.vt.edu/>. During the preparation of this work the author(s) used ChatGPT in order to ensure the grammar and clarity of the manuscript. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

References

- [1] Ultralytics. Yolov11: Performance metrics. <https://docs.ultralytics.com/models/yolo11/#performance-metrics>, 2024. Accessed: March 16, 2025.
- [2] Rahima Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024.
- [3] Ultralytics. Yolov12: Performance metrics. <https://docs.ultralytics.com/models/yolo12/#performance-metrics>, 2024. Accessed: March 16, 2025.
- [4] Yunjie Tian, Qixiang Ye, and David Doermann. Yolov12: Attention-centric real-time object detectors. *arXiv preprint arXiv:2502.12524*, 2025.
- [5] Ultralytics. Detr-rt: Performance metrics. <https://docs.ultralytics.com/models/rtdetr/>, 2024. Accessed: March 16, 2025.
- [6] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.
- [7] Arthur Francisco Araújo Fernandes, João Ricardo Rebouças Dórea, and Guilherme Jordão de Magalhães Rosa. Image analysis and computer vision applications in animal sciences: an overview. *Frontiers in Veterinary Science*, 7:551269, 2020.
- [8] Sarah Morrone, Corrado Dimauro, Filippo Gambella, and Maria Grazia Cappai. Industry 4.0 and precision livestock farming (plf): an up to date overview across animal productions. *Sensors*, 22(12):4319, 2022.
- [9] Wangli Hao, Chao Ren, Meng Han, Li Zhang, Fuzhong Li, and Zhenyu Liu. Cattle body detection based on yolov5-ema for precision livestock farming. *Animals*, 13(22):3535, 2023.
- [10] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE computer society conference on computer vision and pattern recognition. CVPR 2001*, volume 1, pages I–I. Ieee, 2001.
- [11] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [12] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 1440–1448, 2015.
- [13] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. Ssd: Single shot multibox detector. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14*, pages 21–37. Springer, 2016.

- [14] Zhenwei Yu, Yuehua Liu, Sufang Yu, Ruixue Wang, Zhanhua Song, Yinfu Yan, Fade Li, Zhonghua Wang, and Fuyang Tian. Automatic detection method of dairy cow feeding behaviour based on yolo improved model and edge computing. *Sensors*, 22(9):3271, 2022.
- [15] Danqing Xu and Yiquan Wu. Improved yolo-v3 with densenet for multi-scale remote sensing target detection. *Sensors*, 20(15):4276, 2020.
- [16] Abozar Nasirahmadi, Barbara Sturm, Sandra Edwards, Knut-Håkan Jeppsson, Anne-Charlotte Olsson, Simone Müller, and Oliver Hensel. Deep learning and machine vision approaches for posture detection of individual pigs. *Sensors*, 19(17):3738, 2019.
- [17] Bharath Hariharan, Pablo Arbeláez, Ross Girshick, and Jitendra Malik. Hypercolumns for object segmentation and fine-grained localization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 447–456, 2015.
- [18] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.
- [19] Zhaojin Huang, Lichao Huang, Yongchao Gong, Chang Huang, and Xinggang Wang. Mask scoring r-cnn. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6409–6418, 2019.
- [20] Nahian Siddique, Sidike Paheding, Colin P Elkin, and Vijay Devabhaktuni. U-net and its variants for medical image segmentation: A review of theory and applications. *Ieee Access*, 9:82031–82057, 2021.
- [21] Su Myat Noe, Thi Thi Zin, Pyke Tin, and Ikuo Kobayashi. Automatic detection and tracking of mounting behavior in cattle using a deep learning-based instance segmentation model. *Int. J. Innov. Comput. Inf. Control*, 18(1):211–220, 2022.
- [22] Shuqin Tu, Weijun Yuan, Yun Liang, Fan Wang, and Hua Wan. Automatic detection and segmentation for group-housed pigs based on pigms r-cnn. *Sensors*, 21(9):3251, 2021.
- [23] Guoming Li, Yanbo Huang, Zhiqian Chen, Gary D Chesser Jr, Joseph L Purswell, John Linhoss, and Yang Zhao. Practices and applications of convolutional neural network-based computer vision systems in animal farming: A review. *Sensors*, 21(4):1492, 2021.
- [24] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.
- [25] Xu Han, Zhengyan Zhang, Ning Ding, Yuxian Gu, Xiao Liu, Yuqi Huo, Jiezhong Qiu, Yuan Yao, Ao Zhang, Liang Zhang, et al. Pre-trained models: Past, present and future. *AI Open*, 2:225–250, 2021.
- [26] Karim Guirguis, Ahmed Hendawy, George Eskandar, Mohamed Abdelsamad, Matthias Kayser, and Jürgen Beyerer. Cfa: Constraint-based finetuning approach for generalized few-shot object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4039–4049, 2022.

- [27] Chhaya Gupta, Nasib Singh Gill, Preeti Gulia, and Jyotir Moy Chatterjee. A novel finetuned yolov6 transfer learning model for real-time object detection. *Journal of Real-Time Image Processing*, 20(3):42, 2023.
- [28] Thi Thi Zin, Moe Zet Pwint, Pann Thinzar Seint, Shin Thant, Shuhei Misawa, Kosuke Sumi, and Kyohiro Yoshida. Automatic Cow Location Tracking System Using Ear Tag Visual Analysis. *Sensors*, 20(12):3564, January 2020. Number: 12 Publisher: Multidisciplinary Digital Publishing Institute.
- [29] Massachusetts Institute of Technology. Labelme: Image annotation tool. <http://labelme.csail.mit.edu/Release3.0/>, 2023. Accessed: May 23, 2025.
- [30] OpenCV. Cvat: Computer vision annotation tool. <https://www.cvat.ai/>, 2023. Accessed: May 23, 2025.
- [31] Roboflow. Roboflow: Organize, annotate, and improve machine learning datasets. <https://roboflow.com/>, 2023. Accessed: May 23, 2025.
- [32] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [33] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [34] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [35] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.
- [36] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [37] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- [38] Sasha Targ, Diogo Almeida, and Kevin Lyman. Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029*, 2016.
- [39] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [40] Kian Eng Ong, Sivaji Retta, Ramarajulu Srinivasan, Shawn Tan, and Jun Liu. Cattleeyeview: A multi-task top-down view cattle dataset for smarter precision livestock farming. In *2023 IEEE International Conference on Visual Communications and Image Processing (VCIP)*, pages 1–5. IEEE, 2023.
- [41] Eric T. Psota, Ty Schmidt, Benny Mote, and Lance C. Pérez. Long-term tracking of group-housed livestock using keypoint detection and map estimation for individual animal identification. *Sensors*, 20(13):3670, 2020.

- [42] Dataset Ninja. Visualization tools for opencow2020 dataset. <https://datasetninja.com/opencows2020>, may 2024. Accessed: May 21, 2025.
- [43] Kuo Li, Daoerji Fan, Huijuan Wu, and Aruna Zhao. A new dataset for video-based cow behavior recognition. *Scientific Reports*, 14:18702, August 2024.
- [44] Dusty Greif. Ring client api: A typescript api for interfacing with ring devices. <https://github.com/dgreif/ring/tree/main/packages/ring-client-api>, 2023. Accessed: May 23, 2025.
- [45] Ultralytics. Ultralytics datasets documentation. <https://docs.ultralytics.com/datasets/detect/>, 2023. Accessed: May 23, 2025.
- [46] Samuel Sanford Shapiro and Martin B Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3-4):591–611, 1965.
- [47] Howard Levene. Robust Tests for Equality of Variances. In *Contributions to Probability and Statistics: Essays in Honor of Harold Hotelling*, pages 278–292. Stanford University Press, Palo Alto, 1960.
- [48] B. L. Welch. The Generalization of ‘Student’s’ Problem when Several Different Population Variances are Involved. *Biometrika*, 34(1/2):28–35, January 1947.
- [49] H. B. Mann and D. R. Whitney. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1):50–60, March 1947. Publisher: Institute of Mathematical Statistics.
- [50] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics yolo, 2023. If you use this software, please cite it using the metadata from this file.
- [51] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features, August 2019. arXiv:1905.04899 [cs].
- [52] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond Empirical Risk Minimization, April 2018. arXiv:1710.09412 [cs].
- [53] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization, January 2017. arXiv:1412.6980 [cs].
- [54] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems, 2015.

- 1 [55] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen,
2 Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito,
3 Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala.
4 PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information*
5 *Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- 6 [56] Wen Wang, Jing Zhang, Yang Cao, Yongliang Shen, and Dacheng Tao. Towards Data-Efficient Detection
7 Transformers, August 2022. arXiv:2203.09507 [cs].